RICE UNIVERSITY

Data Parallelism and Sharding

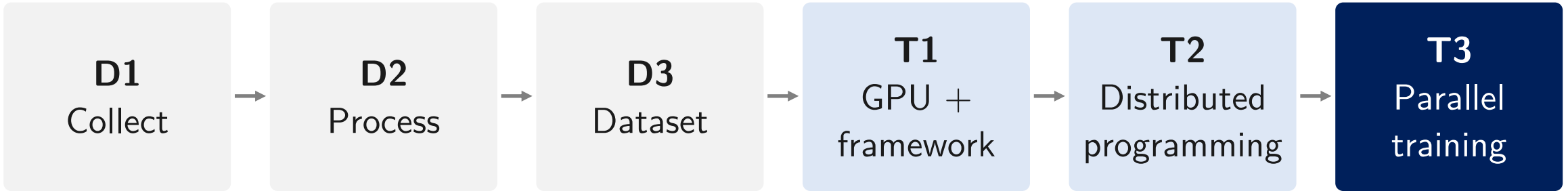
Many copies of one model, one averaged gradient, and how to stop paying for the copies

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 1

Where we are in the pipeline



In our pipeline

Today: train **one model on many GPUs** by splitting the batch. T3 is released after this lecture; **T2 is due Friday**.

Week 11: NCCL collectives and the ring all-reduce cost → today: the all-reduce a training step needs

Learning goals for today

- **Derive** why averaging per-worker gradients equals the gradient of the global batch — and when it does not.
- Estimate the cost of the **one all-reduce** per step with the ring formula.
- Explain how **DDP** buckets gradients and overlaps communication with backward.
- Account for **16 bytes per parameter** of mixed-precision Adam state.
- Compute per-GPU memory and communication for **ZeRO stages 1–3** and **FSDP**.

A puzzle to start

The model
7.5 billion parameters
15 GB of bf16 weights

The cluster
64 GPUs $\times$ 80 GB
$=$ 5,120 GB of memory

Train it with Adam and plain data parallelism. Does it fit?

Answer

No. Every GPU needs $\approx$ **120 GB** of model state — more GPUs do not help. Sharding it (ZeRO-3) needs $\approx$ **1.9 GB** per GPU.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020.

Recap: the cost of an all-reduce

$$\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}$$

each phase: $N - 1$ steps, one chunk of S / N bytes per step

$$T(S, N) = 2(N-1)\ \alpha + 2(N-1)/N \cdot S\ /\ \beta$$

Week 11

Large S: **T** $\approx$ **2S**/β, almost independent of N. Small S: the **latency** term $2(N-1)\alpha$ wins.

Part 1

Data parallelism, derived

Why is averaging gradients the right thing to do?

Replicate the model, split the batch

global batch, B examples

shard 0	shard 1	shard 2	shard 3
↓	↓	↓	↓
GPU 0 · θ forward + backward	**GPU 1** · θ forward + backward	**GPU 2** · θ forward + backward	**GPU 3** · θ forward + backward
g[0]	g[1]	g[2]	g[3]

all-reduce: every GPU gets (g[0] + g[1] + g[2] + g[3]) / 4

every GPU: $\theta \leftarrow$ optimizer step(θ, averaged gradient) $\rightarrow$ copies stay identical

The gradient of a mean is a mean

$$L(\theta) = (1/B)\ \Sigma\ \ell(x;\ \theta) \qquad \text{sum over the global batch}$$

$$g[k] = (1/b)\ \Sigma\ \nabla\ell(x;\ \theta) \qquad \text{sum over shard } k\text{, size } b$$

$$\nabla L = (1/B)\ \Sigma\ b \cdot g[k] \qquad \text{group the sum by shard}$$

$$b = B\ /\ N \quad \Rightarrow \quad \nabla L = (1/N)\ \Sigma\ g[k]$$

Result

Equal shards: **all-reduce SUM, then divide by N**. Exact, not an approximation.

Unequal shards: weight by shard size

$$\nabla L = \Sigma\ (b[k]\ /\ B) \cdot g[k], \qquad B = \Sigma\ b[k]$$

Worker 0
9 examples → weight 9/17

Worker 1
8 examples → weight 8/17

B = 17, as in T3's test

- Plain average (g[0] + g[1]) / 2 → **wrong** gradient.
- Fix: send b[k] · g[k], all-reduce SUM, divide by B.

Watch out

Averaging rank means is only right when every rank has the **same number** of examples.

Sum or mean? Know your loss

Local loss on rank k	Then all-reduce	Then divide by
mean over b examples (equal shards)	SUM of g[k]	N
mean over b[k] examples (unequal)	SUM of b[k] $\cdot$ g[k]	B
sum over b[k] examples	SUM of local gradients	B (for a mean loss)

Watch out

A missing or extra factor of N is a **silent learning-rate change** for SGD. Write the single-process loss first, then match it.

Per-device batch, global batch, learning rate

$$B = N \cdot b \cdot (\text{accumulation steps})$$

- Fix **B**, change N: same gradient, **same** learning rate.
- Fix **b**, add GPUs: B grows — a **different** optimization problem.
- Linear scaling (Goyal et al., 2017): SGD, lr $\propto$ B with warm-up — empirical, limited range.
- Our MLP: B $= 64$; on N $= 4$ GPUs, b $= 16$.

Source: Goyal et al., "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," arXiv:1706.02677, 2017.

When is it not bit-for-bit identical?

Cause	Effect	What to do
Floating-point sum order	last bits differ	compare with a tolerance
BatchNorm	local, not global, statistics	SyncBatchNorm (or none)
Dropout	different random masks	turn off for the check
Sampler padding	repeated examples	check shards cover B once

In our pipeline

T3 compares DP with a single-process reference **within a tolerance** — know why the tolerance is needed.

Check yourself

Two minutes with a neighbour.

- Why may we average per-worker gradients at all? State the condition.
- Two ranks get 30 and 34 of a 64-example batch. What should each rank send?
- Ranks use a **sum** loss but you all-reduce and divide by N. By what factor is the gradient off ($B = 64$, $N = 4$)?

Part 2

One all-reduce per step

How long does agreeing on a gradient take?

Data parallelism by hand

```
for x, y in loader:              # local shard
    opt.zero_grad()
    loss = F.cross_entropy(model(x), y)
    loss.backward()              # local g[k]
    for p in model.parameters():
        dist.all_reduce(p.grad) # SUM, in place
        p.grad /= world_size     # average
    opt.step()                   # same on all
```

- Only **three** new lines.
- One call per tensor: **5** for our MLP.
- Every call pays the latency α.

Watch out

All ranks must call collectives in the **same order** with the **same shapes**, or the job hangs.

Source: PyTorch documentation, torch.distributed.all_reduce (default op = ReduceOp.SUM, in place).

Cost for our MLP

$$S = 34{,}338 \times 4 \text{ bytes} = 137{,}352 \text{ bytes}$$

$$\text{bandwidth term: } 2 \cdot 3/4 \cdot S \;/\; 450 \text{ GB/s} \approx 0.46\ \mu s$$

$$\text{latency term: } 2(N-1)\ \alpha = 6 \times 5\ \mu s \approx 30\ \mu s$$

Memory

Our model's all-reduce is **latency-bound** ($\approx 65\times$). Fewer, larger messages help; more bandwidth does not.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). $\alpha \approx 5$ µs is assumed; use your T2 fit.

Cost for a 7.5-billion-parameter model

$$S = 7.5 \times 10^9 \times 2 \text{ bytes} = 15 \text{ GB of bf16 gradients}$$

Setting	Each rank sends	Bandwidth term
N = 8, one server, NVLink (450 GB/s)	26.2 GB	$\approx$ 58 ms
N = 64, 8 servers, network (50 GB/s)	29.5 GB	$\approx$ 0.59 s

Memory

Large models are **bandwidth-bound**: $\approx 2S/\beta$ per step, and crossing servers costs $\approx 10\times$.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional).

Can the computation hide it?

$$F \approx 6 \cdot \Psi \cdot \text{tokens} = 6 \times 7.5 \times 10^9 \times 16{,}384$$

$$\approx 7.4 \times 10^{14} \text{ FLOPs per GPU per step}$$

$$\text{at } 40\% \text{ of } 989 \text{ TFLOP/s: } \approx 1.86 \text{ seconds}; \quad \text{backward} \approx 1.24 \text{ seconds}$$

Key idea

0.59 s of all-reduce $<$ 1.24 s of backward: it can be **hidden** — if it runs **during** backward.

Source: Kaplan et al., "Scaling Laws for Neural Language Models," 2020 (6 FLOPs per parameter per token); 40% utilization assumed.

Check yourself

- Which term of the ring formula dominates for a 15 GB gradient? For our 137 KB one?
- Our hand-written loop makes 5 all-reduce calls. What do 5 calls cost versus 1 ($N = 4$)?
- What happens to the 64-GPU time if gradients are kept in FP32 instead of bf16?

Part 3

PyTorch DDP

How does a real system make the all-reduce cheap?

DDP in a few lines

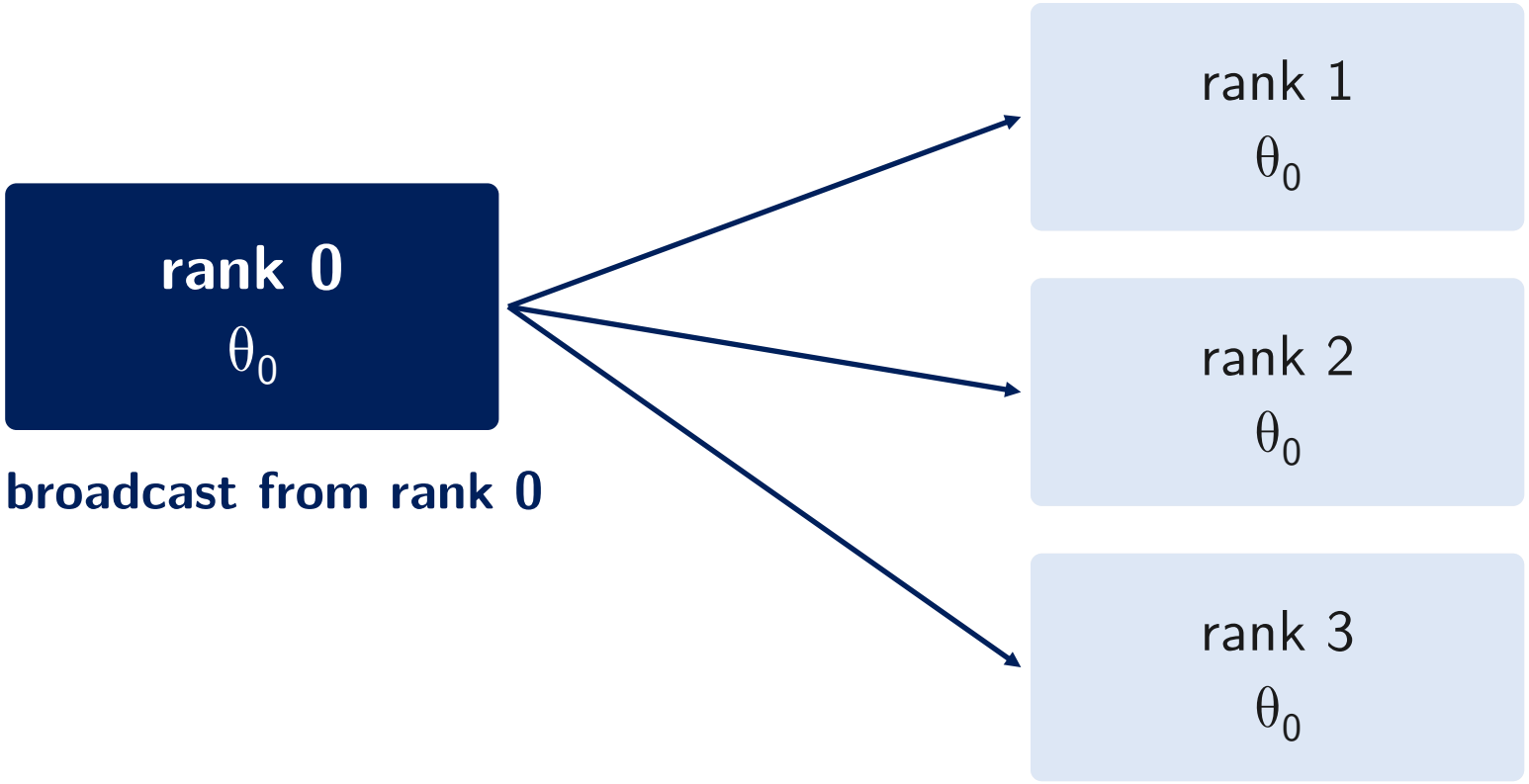
```
# torchrun --nproc_per_node=4 train.py
import os, torch, torch.distributed as dist
from torch.nn.parallel import \
    DistributedDataParallel as DDP

dist.init_process_group("nccl")
rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(rank)
model = DDP(MLP().cuda(), device_ids=[rank])
opt = torch.optim.AdamW(model.parameters())
for x, y in loader:          # DistributedSampler
    opt.zero_grad()
    F.cross_entropy(model(x), y).backward()
    opt.step()
```

- **torchrun**: a process per GPU.
- **DDP**: broadcast + hooks.
- **backward()**: all-reduce, average.
- **Sampler**: disjoint shards.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Step 0: start identical



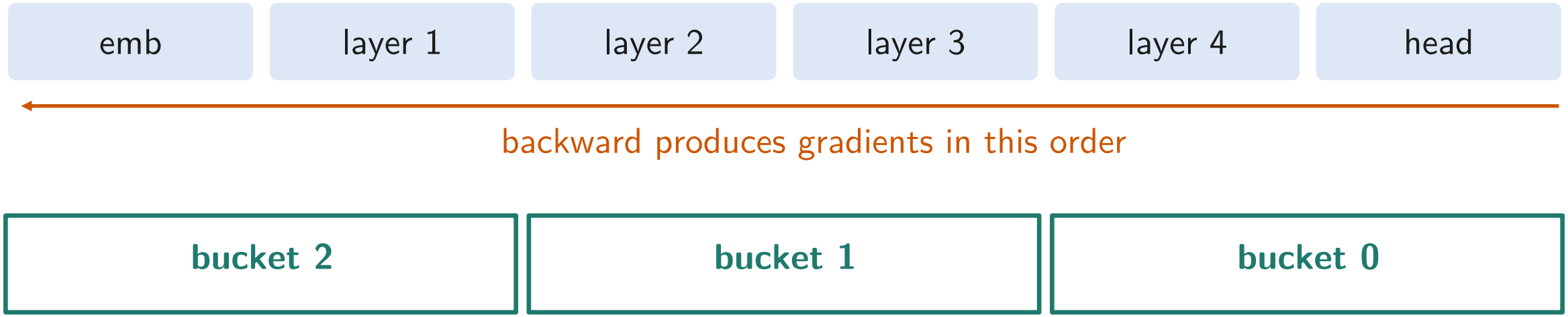
In our pipeline

By hand (T3): same seed on every rank, or `dist.broadcast(p, src=0)` for each parameter.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Gradient bucketing

parameters in registration order



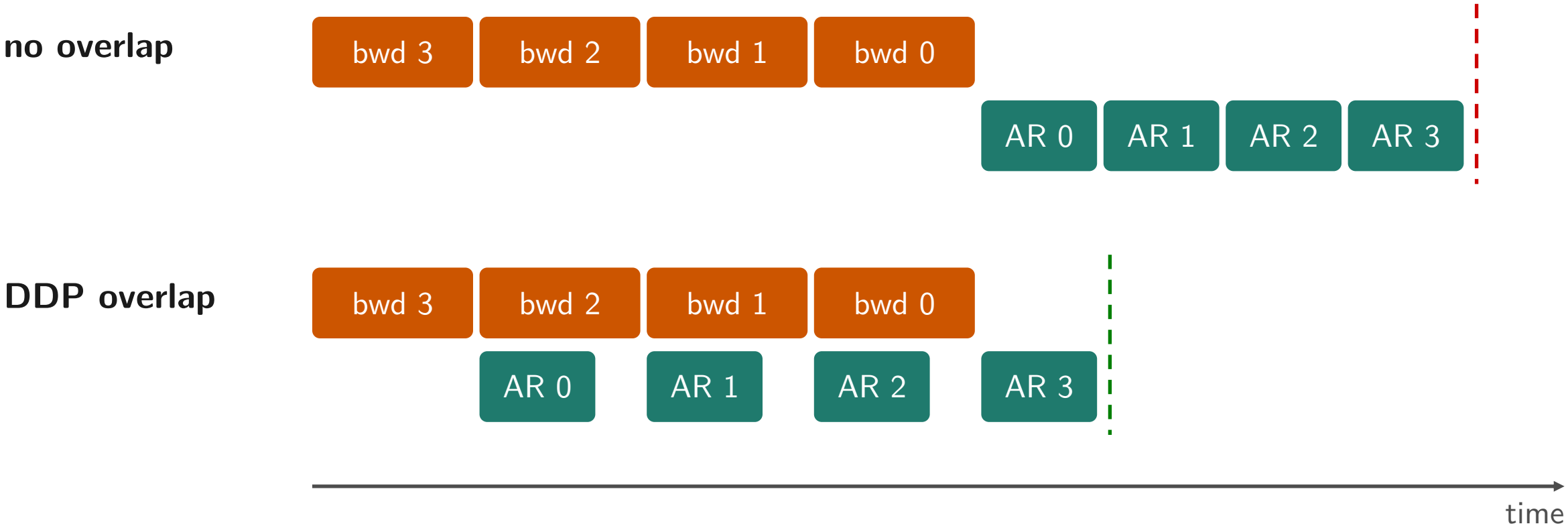
each bucket $\approx$ 25 MiB, one all-reduce when all of its gradients are ready

Memory

7.5B model: 15 GB / 25 MiB $\approx$ **573** buckets. Our MLP: 137 KB $\rightarrow$ **1** bucket, nothing to overlap.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Overlapping all-reduce with backward



Key idea

Step time $\approx$ max(backward, all-reduce) + **one bucket**, instead of the sum.

Why the replicas stay identical

θ_0 identical on every rank (broadcast)

$\theta(t)$ identical $\Rightarrow$ same averaged gradient on every rank

same optimizer, same inputs $\Rightarrow$ $\theta(t+1)$ identical

Watch out

Anything **rank-dependent** in the update (different lr, randomness, an un-reduced gradient) makes the replicas drift apart.

DDP options you will meet

Option	Default	Use it for
`bucket_cap_mb`	25 MiB	fewer calls vs more overlap
`no_sync()`	—	gradient accumulation
`find_unused_parameters`	False	models with branches that skip parameters
`gradient_as_bucket_view`	False	save one copy of the gradients

Watch out

DDP fixes the **time** of data parallelism. It does nothing for **memory**: every rank still holds everything.

Source: PyTorch documentation, torch.nn.parallel.DistributedDataParallel and "DDP notes"; Li et al., VLDB 2020.

Check yourself

- What does the DDP constructor send, from where, and why?
- Why are buckets formed in **reverse** order of the parameters?
- Will DDP's overlap speed up our 34,338-parameter MLP? Why or why not?

Part 4

ZeRO and FSDP

Every GPU holds the same 16 bytes per parameter. Why?

Where the memory goes: 16 bytes per parameter

16-bit parameters	2Ψ
16-bit gradients	2Ψ
FP32 master parameters	4Ψ
Adam momentum (FP32)	4Ψ
Adam variance (FP32)	4Ψ

optimizer state, K = 12

$$\text{total} = (2 + 2 + 12)\ \Psi = 16\Psi \text{ bytes}$$

$\Psi = 7.5 \times 10^9$: **120 GB**

our MLP: $\approx$ **0.55 MB**

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 3.1.

Data parallelism replicates all of it



blue: 16-bit params 2Ψ · orange: 16-bit grads 2Ψ · green: optimizer 12Ψ

Key idea

N identical copies of the optimizer state — **75%** of the memory — that each rank could own one N-th of.

ZeRO stages 1, 2 and 3



one GPU of N = 4, drawn to scale; shards shrink by N

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

Per-GPU memory, with numbers

Stage	Shards	Per GPU	$\Psi = 7.5$B, N = 64
DP	nothing	16Ψ	120 GB
ZeRO-1	optimizer state	$4\Psi + 12\Psi$ / N	31.4 GB
ZeRO-2	+ gradients	$2\Psi + 14\Psi$ / N	16.6 GB
ZeRO-3	+ parameters	16Ψ / N	1.9 GB

Memory

Stages 1–2 approach 4Ψ and 2Ψ; only **stage 3** keeps falling as 1/N. Activations are extra.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Figure 1.

ZeRO-1 and -2 communicate like DP

DP: all-reduce = reduce-scatter + all-gather $\rightarrow$ $\Psi + \Psi = 2\Psi$

ZeRO-2: reduce-scatter gradients $\rightarrow$ Ψ (each rank: its $1/N$, summed)

update own $1/N$ of θ with own $1/N$ of optimizer state

all-gather updated 16-bit parameters $\rightarrow$ Ψ $\Rightarrow$ total 2Ψ

Check

Stages 1 and 2: same $\mathbf{2\Psi}$ per rank as plain data parallelism — memory saved at no extra traffic.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 7.2.

ZeRO-3 costs 1.5× the communication

$$\text{forward: all-gather parameters, layer by layer} \rightarrow \Psi$$

$$\text{backward: all-gather them again (dropped after use)} \rightarrow \Psi$$

$$\text{gradients: reduce-scatter} \rightarrow \Psi$$

$$\text{total } 3\Psi = 1.5 \times 2\Psi$$

Watch out

Memory ÷ N for **+50%** communication. Worth it when the model does not fit otherwise.

Source: Rajbhandari, Rasley, Ruwase & He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. Section 7.2.

FSDP: PyTorch's ZeRO-3

forward



backward



AG: all-gather parameters · RS: reduce-scatter gradients

Key idea

Peak memory $\approx 16\Psi/N +$ **one or two unsharded units**. Prefetching the next AG hides it behind compute.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

FSDP in code

```
from torch.distributed.fsdp import fully_shard

model = Transformer().cuda()
for block in model.layers:
    fully_shard(block)     # one sharding unit
fully_shard(model)         # root: the rest
opt = torch.optim.AdamW(model.parameters())

for x, y in loader:        # same loop as DDP
    opt.zero_grad()
    loss_fn(model(x), y).backward()
    opt.step()
```

- Shard **bottom-up**: blocks, then root.
- Optimizer built **after** sharding: state is 1/N.
- Parameters are **DTensors** split on dim 0.

Source: PyTorch documentation, torch.distributed.fsdp.fully_shard (FSDP2); Zhao et al., "PyTorch FSDP," VLDB 2023.

Check yourself

- List the five pieces of the 16 bytes per parameter.
- ZeRO-2, $N = 8$, $\Psi = 1$ billion: how many GB of model state per GPU?
- Why does stage 3 move $1.5\times$ the data of plain data parallelism?

IN-CLASS BUILD

Check an update, count memory

Work in pairs. Part A is the core of T3; Part B is pencil and Python.

Exercise A: check a data-parallel update

```
torch.manual_seed(0)          # same init
model = MLP(); ref = copy.deepcopy(model)
xb, yb = shard(X, Y, rank, world)   # disjoint
F.cross_entropy(model(xb), yb).backward()
for p in model.parameters():
    dist.all_reduce(p.grad)
    p.grad /= ____                  # (1)
F.cross_entropy(ref(X), Y).backward()  # 1 proc
for p, q in zip(model.parameters(),
                ref.parameters()):
    torch.testing.assert_close(p.grad, q.grad)
```

QUESTIONS

(1) Fill in the blank.

(2) 17 examples on 2 ranks, 9 and 8: what changes?

(3) Use DDP instead. Does it still pass with 9 and 8?

Exercise B: account for memory

Stage	Formula	Model state per GPU	Fits in 60 GB?
DP	16Ψ		
ZeRO-1	$4\Psi + 12\Psi$ / N		
ZeRO-2	$2\Psi + 14\Psi$ / N		
ZeRO-3	16Ψ / N		

GIVEN

$\Psi = 13 \times 10^9$, N $= 8$, 80 GB GPUs, 20 GB kept for activations. GB $= 10^9$ bytes.

STRETCH

Which stage would you pick, and what does it cost in communication?

Solutions

- (1) `world` — equal shards.
- (2) Send b[k] $\cdot$ g[k]; divide by B $= 17$.
- (3) DDP averages by N: **fails** for $9 + 8$ unless you rescale the loss.

```
def per_gpu(psi, n):        # bytes
    return {
        "DP": 16 * psi,
        "ZeRO-1": 4 * psi + 12 * psi / n,
        "ZeRO-2": 2 * psi + 14 * psi / n,
        "ZeRO-3": 16 * psi / n}

for k, v in per_gpu(13e9, 8).items():
    print(k, v / 1e9, "GB")
```

	DP	ZeRO-1	ZeRO-2	ZeRO-3
GB	208	71.5	48.75	26
fits?	no	no	yes	yes

How to test it

```
# torchrun --nproc_per_node=2 check.py
dist.init_process_group("gloo")   # CPU
rank = dist.get_rank()
world = dist.get_world_size()
```

In our pipeline

T3, part 1: hand-written DP vs a single-process reference vs DDP, on D3 data.

- **gloo on CPU** first, then NCCL on GPUs.
- Compare **gradients and parameters** after `opt.step()`.
- Use `assert_close` defaults (float32: rtol 1.3e-6, atol 1e-5).
- Test an **uneven** batch of 17.
- Delete the division: the test must **fail**.

Source: PyTorch documentation, torch.testing.assert_close (default tolerances by dtype).

Key takeaways

- Gradient of a mean = mean of gradients: **all-reduce SUM, divide by N** — for equal shards only.
- Unequal shards: weight by **b[k]/B**. Sum vs mean losses: a silent factor of N.
- One all-reduce per step: $\approx$ **2S**/β for big models, **latency-bound** for ours.
- **DDP**: broadcast once, bucket gradients, overlap all-reduce with backward.
- Mixed-precision Adam: **16 bytes/parameter**, replicated on every GPU by plain DP.
- **ZeRO-1/2** shard it at DP's traffic; **ZeRO-3 / FSDP** shards everything for 1.5×.

Next time, and readings

When	What
Friday	T2 due (distributed programming)
After this lecture	T3 released: DP vs single-process reference vs DDP, on D3 data
Lecture 2 (this week)	Tensor and model parallelism: splitting one layer across GPUs
Week 11 (review)	NCCL collectives; multi-server communication

READING

Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020 — Sections 1, 3, 5 and 7.